UNIVERSITÉ PARIS-DAUPHINE – PSL
CENTRE DE RECHERCHE EN MATHÉMATIQUES DE LA DÉCISION

LE CONSEILLER ENNEIGÉ - MON BIBLIOTHÉCAIRE IA

AXELLE MERIC

COURSE SUPERVISOR:

THÉO LOPES-QUINTAS
BPCE PAYMENT SERVICES

GENERATIVE AI
ACADEMIC YEAR 2025/2026

Contents

1 Introduction **2**

2 Collecte et enrichissement des données (Web Scraping) **2**

3 Recherche Sémantique (RAG) et Base Vectorielle **4**
3.1 Choix du modèle d'Embeddings 4
3.2 Filtrage Hybride et Exploitation des Métadonnées 4

4 L'Agent Intelligent (LangGraph & MistralAI) **5**
4.1 Choix du Modèle de Langage (LLM) : De l'embarqué au Cloud 5
4.2 Orchestration par Graphe d'États (LangGraph) 5

5 Interface Utilisateur et Identité Visuelle (Streamlit) **6**

6 Conclusion et Perspectives d'évolution **8**

1 Introduction

Dans le cadre du cours de Generative AI, nous avons été amenés à concevoir une application exploitant les technologies récentes de Traitement du Langage Naturel (NLP) et d'Intelligence Artificielle générative.
L'objectif de ce projet, baptisé *Le Conseiller Enneigé*, est de proposer un assistant conversationnel intelligent capable d'interagir avec la Pile à Lire (PAL) d'un utilisateur. En combinant des techniques d'enrichissement de données (Web Scraping), de recherche sémantique (RAG - Retrieval-Augmented Generation) et d'orchestration d'agents (LangGraph), cette application offre des recommandations de lecture ultra-personnalisées de manière naturelle et fluide.
Ce rapport détaille la méthodologie adoptée, l'architecture technique du système, ainsi que les choix d'implémentation réalisés pour aboutir à l'application finale.

2 Collecte et enrichissement des données (Web Scraping)

La première étape du projet a consisté à construire une base de connaissances riche et structurée. Le jeu de données initial provient de **Livraddict**, un réseau social littéraire et une application de gestion de bibliothèque virtuelle. Cette plateforme offre la possibilité à ses utilisateurs d'exporter leur « Pile à Lire » (PAL) sous forme de fichier CSV.
Cependant, bien que cet export contienne de précieuses métadonnées (titre, auteur, genre, nombre de pages, note moyenne, etc.), il y manque un élément crucial pour notre système de recommandation : le résumé de l'œuvre. Sans ce texte narratif, l'agent conversationnel serait incapable d'effectuer une recherche sémantique (RAG) pertinente sur l'intrigue ou les thèmes des livres.
Pour pallier ce manque et s'adapter à la diversité de ma Pile à Lire, qui contient des ouvrages aussi bien en français qu'en anglais, un script automatisé d'enrichissement de données (*Web Scraping*) a été développé en Python. Conçu à l'aide de la bibliothèque `BeautifulSoup` et du module `requests`, ce script vise à maximiser le taux de complétion de notre base en interrogeant trois plateformes distinctes selon un ordre de priorité défini (en cascade). Pour chacune d'elles, la méthode d'extraction a été spécifiquement adaptée :

- **Livraddict (Priorité 1)** : C'est la source principale pour les livres. Pour éviter toute erreur de recherche (comme les homonymes de titres ou les tomes de sagas mal indexés), une stratégie de navigation précise a été mise en place via la bibliothèque `BeautifulSoup`. Au lieu de chercher le livre directement, le script recherche d'abord la page de l'auteur, navigue vers sa bibliographie complète, puis cherche le titre exact dans cette liste pour accéder à la bonne page et en extraire le résumé (voir Figures 1 et 2). Dans le cas spécifique d'un ouvrage écrit par un « collectif », le script bascule intelligemment sur une recherche par titre seul.

- **Google Books (Priorité 2)** : C'est la méthode la plus simple et la plus directe techniquement. Google mettant à disposition une API publique, il n'y a pas besoin de faire du *web scraping* complexe. Le script effectue une requête HTTP croisée "titre + auteur" et récupère directement la description de l'ouvrage au format JSON structuré.

- **Goodreads (Priorité 3)** : C'est l'ultime alternative, indispensable pour la littérature anglophone. Afin de garantir l'exactitude du résultat, le script y effectue d'abord une recherche

ciblée sur le titre du livre, puis parcourt la liste des résultats en effectuant une double vérification : il croise chaque résultat avec le nom de l'auteur pour s'assurer de sélectionner le bon ouvrage. Une fois la correspondance validée, un web scraping avancé (extraction de méta-données JSON-LD et de balises HTML spécifiques) permet de récupérer le résumé complet.

Ce processus automatisé en cascade a permis de transformer une simple liste tabulaire en une véritable base de connaissances textuelle prête à être vectorisée.

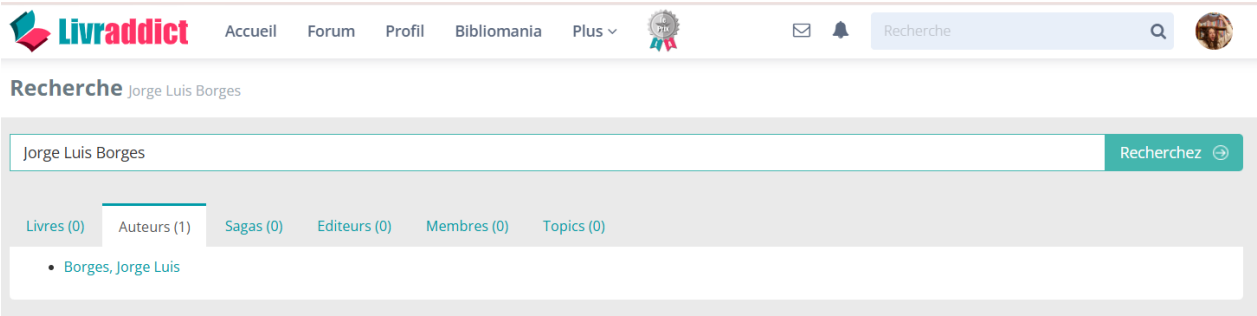

Figure 1: Exemple de recherche d'auteur ciblée sur le site Livraddict.

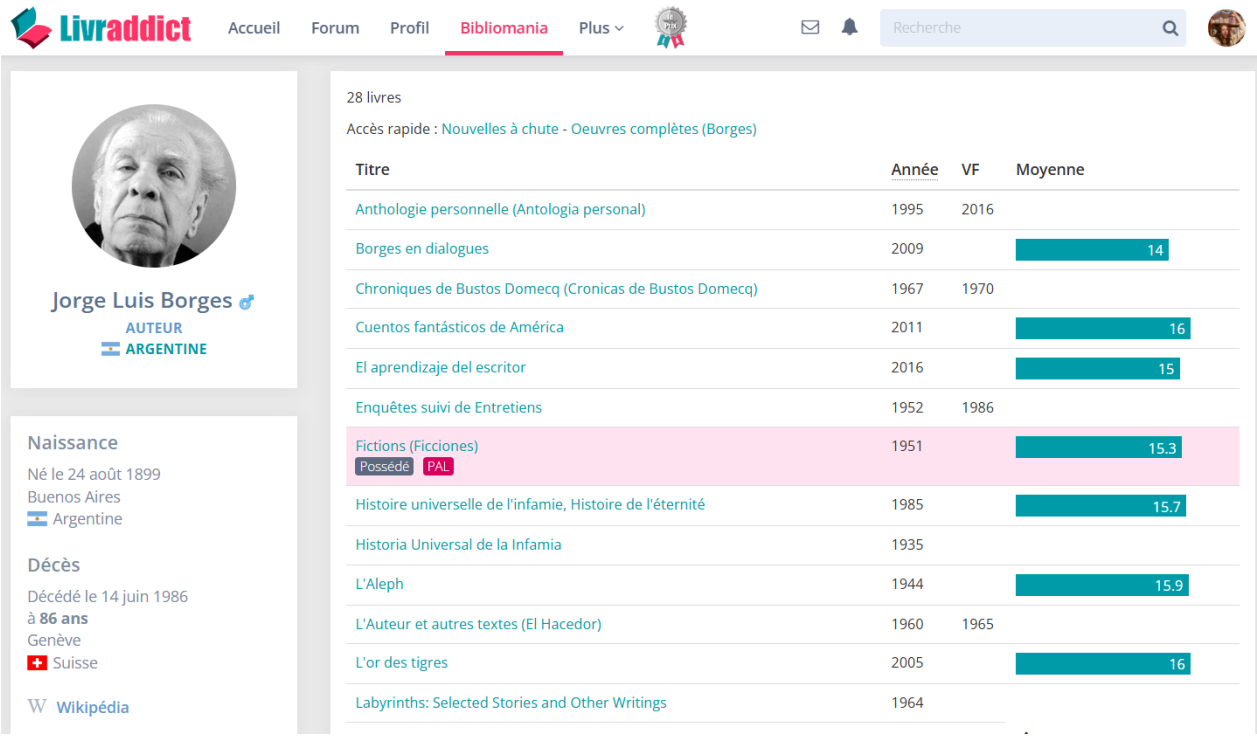

Figure 2: Parcours de la bibliographie sur la page de l'auteur pour garantir l'exactitude du livre sélectionné.

3 Recherche Sémantique (RAG) et Base Vectorielle

Une fois la base de données textuelles constituée, l'enjeu était de permettre à l'agent intelligent de « comprendre » le sens des résumés pour répondre aux requêtes en langage naturel de l'utilisateur. Pour ce faire, nous avons mis en place une architecture RAG (*Retrieval-Augmented Generation*) s'appuyant sur la vectorisation des documents (Titre, Auteur et Résumé).

3.1 Choix du modèle d'Embeddings

La transformation du texte en vecteurs mathématiques (Embeddings) est une étape critique qui détermine la qualité de la recherche. Au cours de la phase de prototypage, plusieurs modèles ont été testés et évalués en utilisant diiférentes distances dont la distance cosinus pour mesurer la similarité entre la requête et les documents :

- **Tests initiaux** : Les modèles `all-MiniLM-L6-v2` (HuggingFace) et `nomic-embed-text` (via Ollama) ont d'abord été éprouvés. Bien que très rapides, ces modèles se sont révélés trop « petits » et insuffisamment entraînés pour capturer la complexité sémantique de certains résumés littéraires. Les résultats remontés manquaient cruellement de pertinence face à des requêtes nuancées.
- **Modèle final retenu** : Pour pallier ce problème et s'adapter parfaitement à la nature bilingue (français/anglais) de la Pile à Lire, le choix final s'est porté sur `paraphrase-multilingual-MiniLM-L12-v2` (HuggingFace). Plus profond (12 couches au lieu de 6), ce modèle offre un excellent compromis entre performances de calcul et finesse de compréhension sémantique multilingue.

3.2 Filtrage Hybride et Exploitation des Métadonnées

La similarité sémantique pure atteint rapidement ses limites dans un cas d'usage comme celui d'une bibliothèque : un utilisateur peut chercher un livre traitant de « magie » (critère sémantique), mais exiger qu'il fasse « moins de 300 pages » ou qu'il ait été « acheté l'année dernière » (critères stricts).
Pour répondre à ce besoin, une approche de recherche hybride a été implémentée. Cette architecture n'a été rendue possible que grâce à la richesse structurelle du fichier CSV initial exporté depuis Livraddict. Lors de la création des documents pour FAISS, plutôt que de ne conserver que le texte, l'ensemble de ces informations tabulaires a été minutieusement nettoyé puis injecté dans les métadonnées (`metadata`) de chaque objet vectorisé.
Ainsi, l'utilisateur a la possibilité de croiser sa recherche avec un grand nombre de filtres stricts :

- **Filtres textuels** : Auteur, Genre littéraire, Éditeur, Public visé (Lectorat), Langue.
- **Filtres numériques** : Nombre de pages (minimum et maximum), Année de publication (min/max), Note moyenne minimale.
- **Filtres temporels** : Date d'achat (avant ou après une date spécifique).

La mise en place de cette fonction de filtrage à la volée (`filtre_faiss`) a représenté un véritable défi technique, nécessitant un important travail de préparation des données en amont :

- **Conversion temporelle** : Pour permettre à l'algorithme de comparer mathématiquement si un livre a été acheté « avant » ou « après » une certaine date, il a fallu convertir des dates textuelles au format français (JJ/MM/AAAA) en *timestamps* UNIX (un nombre de secondes).
- **Fusion et nettoyage** : L'export Livraddict séparant historiquement les genres littéraires sur deux colonnes distinctes, une fusion logicielle a été nécessaire pour créer un champ interrogeable de manière fluide. De même, un typage rigoureux a été appliqué pour convertir les notes en flottants et gérer les valeurs manquantes (`NaN`) afin d'éviter tout crash lors de l'exécution.

En pratique, l'outil FAISS est configuré pour pré-sélectionner un très large bassin de documents via la similarité (`fetch_k=1000`) avant d'appliquer ces filtres intraitables et de retourner les candidats finaux correspondants à la fois à l'intention sémantique et aux contraintes techniques de l'utilisateur.

4 L'Agent Intelligent (LangGraph & MistralAI)

L'intelligence cognitive de l'application, chargée de comprendre l'utilisateur et d'orchestrer les recherches, repose sur une architecture d'agent avancé.

4.1 Choix du Modèle de Langage (LLM) : De l'embarqué au Cloud

Lors de la phase de prototypage, la première volonté a été d'utiliser des modèles de langage exécutés localement via le moteur **Ollama** (comme `llama3` ou `gemma`). Bien que cette approche en local soit séduisante pour la confidentialité, elle s'est rapidement heurtée aux limites physiques de la machine :

- **Goulet d'étranglement matériel** : Le téléchargement des poids des modèles saturait l'espace de stockage, et leur exécution monopolisait la mémoire vive (RAM).
- **Latence critique** : Les temps d'inférence (génération de la réponse) étaient excessivement longs, dégradant fortement l'interactivité requise pour un assistant conversationnel (chatbot).

Face à ces contraintes, l'architecture a pivoté vers une solution Cloud via l'API de **MistralAI**. Le modèle `mistral-small-latest` a été sélectionné pour sa légèreté, sa rapidité fulgurante, et surtout son excellente capacité de raisonnement et de formatage en français, déportant ainsi toute la charge de calcul sur des serveurs distants.

4.2 Orchestration par Graphe d'États (LangGraph)

Pour éviter que le LLM ne « rêve » (hallucinations) ou ne lance des recherches erratiques, son comportement a été strictement canalisé à l'aide de LangGraph. Ce framework permet de modéliser l'agent sous la forme d'une machine à états finis, garantissant un flux d'exécution prévisible et sécurisé.
Le graphe d'états de notre Bibliothécaire IA est structuré autour de quatre nœuds fondamentaux :

1. **Nœud de Planification (Extraction JSON rigoureuse)** : L'agent reçoit la requête en langage naturel de l'utilisateur. Son unique rôle à ce stade n'est pas de discuter, mais d'extraire les critères de recherche. Pour garantir une fiabilité absolue lors du transfert vers FAISS, nous avons couplé le LLM à un `PydanticOutputParser`. L'IA est ainsi forcée de formater sa réponse sous la forme d'un objet JSON strict, mappant parfaitement nos métadonnées. *Subtilité d'implémentation* : Si l'utilisateur ne formule aucune demande sur l'intrigue (ex: « Je veux une romance de 300 pages »), le modèle a pour instruction stricte d'assigner la valeur par défaut `"livre"` au champ de recherche sémantique.

2. **Nœud de Validation (Human-in-the-loop)** : C'est une fonctionnalité clé pour la transparence de l'IA. Avant d'exécuter la moindre recherche coûteuse, l'agent interrompt son cycle et présente les critères extraits à l'utilisateur sous forme de liste à puces. Le système se met en pause (*attente_validation*) et demande explicitement une confirmation (Oui/Non). Si l'utilisateur refuse, le graphe annule la recherche et invite à reformuler.

3. **Nœud d'Exécution (Appel d'Outil)** : Une fois l'approbation humaine reçue, le graphe transmet le JSON de critères à notre outil de recherche hybride (`@tool`). C'est ici que la base FAISS est interrogée. La gestion de la valeur par défaut `"livre"` y prend tout son sens : le code détecte cette valeur générique et désactive le seuil d'élimination basé sur la distance. Cela permet aux requêtes purement basées sur les filtres stricts (métadonnées) d'aboutir avec succès, sans être faussées par le calcul de similarité textuelle.

4. **Nœud de Synthèse (Génération de la Réponse)** : Le LLM reçoit en contexte les résumés et les titres des livres réellement trouvés par FAISS. Un *prompt* (consigne) lui interdit formellement d'inventer des œuvres. Il adopte alors son « persona » de bibliothécaire chaleureux pour présenter les résultats à l'utilisateur de manière naturelle et argumentée.

Cette architecture modulaire garantit que l'agent ne propose que des livres réellement présents dans la Pile à Lire de l'utilisateur, tout en offrant une expérience interactive fluide.

5 Interface Utilisateur et Identité Visuelle (Streamlit)

Pour que ce projet dépasse le simple cadre de l'expérimentation algorithmique et devienne un véritable produit utilisable, une interface web interactive a été développée avec le framework **Streamlit**.
Plutôt que de se contenter du design standard et minimaliste de la bibliothèque, un travail important d'intégration CSS et de personnalisation (*branding*) a été réalisé pour que l'application reflète l'identité de mon futur profil littéraire sur les réseaux sociaux : **« Le livre enneigé »**. L'assistant a ainsi été logiquement baptisé *Le conseiller enneigé*, accompagné de sa mascotte en photo (« Piou Piou ») intégrée dès l'en-tête de la page.
L'interface a été intégralement repensée pour offrir une expérience immersive et esthétique :

- **Personnalisation CSS avancée** : Le code injecte des règles de style complexes pour outrepasser les thèmes par défaut. Une charte graphique harmonieuse a été appliquée, s'articulant autour de deux nuances de violet (un violet foncé pour les textes et titres, et un violet pastel clair pour les cadres de saisie et de réponse).

- **Habillage immersif** : Grâce à un encodage des images en base64 injecté dans le CSS (`fixed-img-left` et `fixed-img-right`), de véritables photographies de mes propres bibliothèques physiques encadrent la zone de discussion de part et d'autre de l'écran (voir Figure 3).
- **Avatars sur-mesure** : Les icônes génériques de Streamlit ont été remplacées par des images personnalisées pour différencier clairement les interactions entre l'utilisatrice et l'agent. Poussant la démarche de *personal branding* jusqu'au bout, l'avatar me représentant dans le chat est ma véritable photo de profil actuelle sur la plateforme Livraddict (d'où proviennent les données), et servira d'identité visuelle officielle pour mes futurs réseaux sociaux. L'agent, quant à lui, est incarné par une photographie chaleureuse de ma propre bibliothèque baignée d'un rayon de soleil, renforçant la métaphore du bibliothécaire intime et personnel. La mascotte du compte, « Piou Piou », trône quant à elle fièrement dans l'en-tête de l'application pour asseoir l'identité visuelle.
- **Gestion de l'état** : L'historique de la conversation et les étapes de validation (*Human-in-the-loop*) sont gérés de manière persistante grâce au `st.session_state`.

Performance et pertinence : Il est crucial de souligner que malgré cette refonte graphique complète (chargement d'images haute résolution en base64) et la complexité des opérations en arrière-plan (graphe d'états, appels API Mistral, recherche vectorielle FAISS et filtres sur métadonnées), l'application conserve une fluidité exemplaire. Les temps de réponse restent extrêmement rapides, et l'assistant délivre des recommandations hautement pertinentes de manière quasi instantanée (voir Figure 4).

Figure 3: Interface globale affichant la charte graphique violette et les bibliothèques intégrées en arrière-plan.

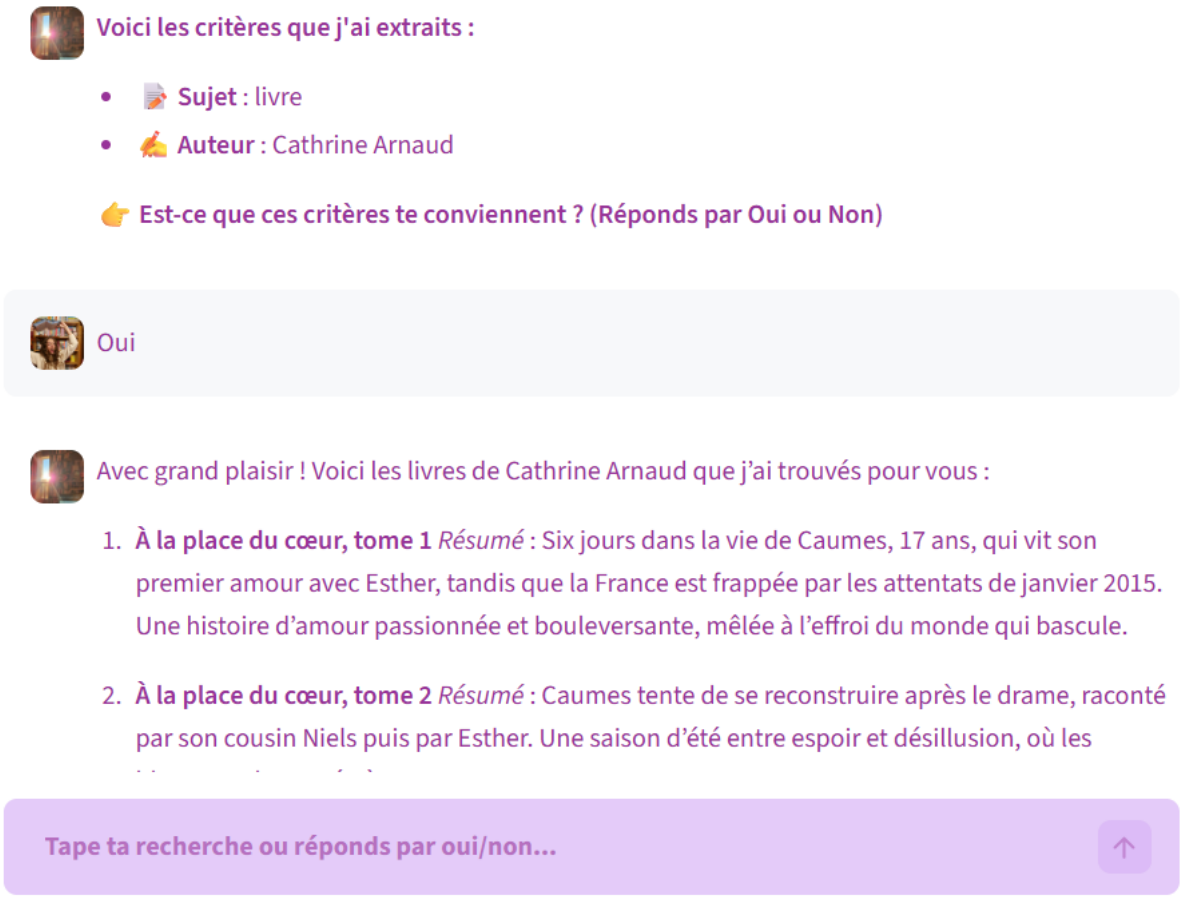

Figure 4: Exemple d'interaction montrant l'extraction des critères, la validation humaine, et la réponse pertinente de l'agent.

6 Conclusion et Perspectives d'évolution

Le projet *Le Conseiller Enneigé* a permis de concevoir, de bout en bout, une application d'Intelligence Artificielle générative robuste, appliquée à la recommandation littéraire. En partant d'une base de données tabulaire incomplète, nous avons déployé un pipeline d'enrichissement de données (*Web Scraping*) pour constituer un corpus textuel riche. Le défi de l'interrogation a été relevé grâce à une architecture RAG hybride (FAISS couplé à un filtrage strict sur métadonnées), propulsée par des *embeddings* multilingues. Enfin, l'orchestration via LangGraph et MistralAI a permis de canaliser l'IA, garantissant l'absence d'hallucinations tout en intégrant un protocole de validation humaine (*Human-in-the-loop*). Le tout a été encapsulé dans une interface Streamlit hautement personnalisée, reflétant mon identité visuelle.
Ce prototype fonctionnel pose des bases algorithmiques solides. Conçu de manière modulaire, le système est d'ores et déjà prêt à accueillir plusieurs extensions techniques majeures que j'ai prévu d'implémenter lors des prochaines itérations :

- **Apprentissage des préférences (Benchmark)** : La prochaine étape consistera à ingérer le CSV de mes livres déjà lus sur Livraddict. Ce fichier contenant mes notes personnelles sur 20,

il servira de benchmark de vérité terrain (*ground truth*) pour évaluer et affiner la pertinence du moteur de recommandation en fonction de mes goûts réels.

- **Gestion d'inventaire et localisation spatiale** : Ma Pile à Lire étant particulièrement volumineuse et physiquement répartie sur deux sites géographiques distincts (Paris et Gap), les métadonnées de la base vectorielle seront enrichies par des coordonnées spatiales. L'agent sera ainsi capable de m'indiquer non seulement quel livre lire, mais aussi dans quelle ville et sur quelle étagère précise il se trouve.
- **Affichage visuel dynamique** : Possédant déjà une base de données locale répertoriant les photographies de chaque livre de ma PAL, l'outil de génération de réponse de l'agent sera mis à jour pour extraire et afficher la véritable photo de la couverture du livre recommandé directement dans le chat Streamlit, facilitant ainsi sa recherche visuelle sur l'étagère.
- **Gestion de la chronologie des sagas** : L'intégration d'une logique permettant à l'agent de comprendre les relations d'antériorité entre les tomes d'une même série. Bien que la modélisation de cette arborescence représente un défi technique complexe, c'est une fonctionnalité indispensable pour éviter qu'un bibliothécaire ne recommande le tome 3 d'une saga avant le tome 1.

En définitive, ce projet démontre qu'il est possible d'allier l'ingénierie logicielle avancée du NLP à la création d'un outil quotidien véritablement utile, esthétique et hautement évolutif.

Remerciements

Je tiens à adresser un remerciement tout particulier à Théo pour son aide précieuse, sa patience et ses conseils avisés qui m'ont grandement aidée dans la conception et le développement de cette interface.